The goal of this first task is to take images with different settings of a camera to create pictures with perspective projection and with orthographic projection. Both pictures should cover the same piece of the scene. You can take pictures of real places or objects (e.g. your furniture).
To create pictures with orthographic projection you can do two things: 1) use the zoom of the camera, 2) crop the central part of a picture. You will have to play with the distance between the camera and the scene, and with the zoom or cropping so that both images look as similar as possible, only differing in the type of projection. Submit the two pictures and clearly label parts of the images that reveal their projection types. **[2Marks]**

Second task is to write a function that applies a *rigid-body transform* to a given 3D point, i.e., a rotation and a translation. Specifically, we will implement an operation that we will need all the time: transforming a point from *camera coordinates* to *world coordinates* and vice versa. First, let's talk about rigid-body transforms. Given a vector $\mathbf{v}$ that we want to transform, a rigid-body transform is formally defined as: $\mathbf{v}'=\mathbf{R}\mathbf{v}+\mathbf{t}$, with a rotation matrix $\mathbf{R}\in SO(3)$, and a translation $\mathbf{t}\in\mathbb{R}3$.

Instead of breaking up this transformation into a matrix-vector product and a sum, we may instead operate on *homogeneous coordinates*. In homoegenous coordinates, we simply extend vectors and points in $\mathbb{R}3$ by a fourth coordinate. *Points* are extended by a 1, as a translation should effect them. *Directions* are extended by a 0, as they shouldn't be effected by a translation. I.e., for all the points in our pointcloud, we simply append a fourth coordinate, a 1, to all of them.

This allows us to compactly represent the full transform as a single matrix of the following form:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix}$$

We can then write a rigid-body transform as a single matrix-vector product as follows:

$$\mathbf{v}' = \mathbf{T}\mathbf{v}$$

Since $\mathbf{v}$ has a fourth component that is a 1, the first three components of $\mathbf{v}$ are rotated, and the fourth component adds the translation part of $\mathbf{T}$ to the result. This has another perk: if we want to revert this transformation, we can achieve this by simply inverting the matrix $\mathbf{T}$

$\mathbf{v} = \mathbf{T}^{-1}\mathbf{v}'$.

Implement three functions: one to homogenize *points* (append a 1 to the coordinates), one to homogenize *vectors* (append a 0 to the coordinates), and one to apply a rigid-body transform to homogenized points / vectors. [3 **Marks]**

For this exercise you will use the Scale-Invariant Feature Transform (SIFT) for matching. You will extract SIFT features from two images and use them to find feature correspondences and solve for the affine transformation between them. Please include code under each question. You are allowed to use external code for SIFT keypoint and descriptor extraction

Compute SIFT features for reference.png, test.png, and test2.png. Please visualize the detected keypoints on the image. By visualize we mean: plot the image, mark the center of each keypoint, and draw either a circle or rectangle to indicate the scale of each keypoint. For clarity, please plot only 100 keypoints.

Given the extracted features on reference.png and test.png, implement a KNN matching algorithm. Visualize the top (best) 3 correspondences. Do the same for test2.png

Use the top 3 correspondences from above to solve for the affine transformation between the features in the two test images. Visualize the affine transformation. Do this visualization by taking the four corners of the test image, transforming them via the computed affine transformation to the points in the test2 image, and plotting those transformed points. Please also plot the edges between the points to indicate the parallelogram. [5 **Marks]**